

Lume

Manual técnico do projeto

Plataforma web para organizar e ler PDF e DOCX, com capas automáticas, descrição por IA e leitor próprio. Django, Java, JavaScript, PostgreSQL, Redis, WebSockets e Docker.

Copyright (©) 2026 Clara A. Portes

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 4

- Para que serve 4
- O que o Lume faz 4
- Tecnologias 4
- Identidade visual 5

Como rodar 6

- Requisitos 6
- Primeira execução 6
- Contas de demonstração 7
- Comandos do dia a dia 7
- Usar de verdade 7
- Rodar sem Docker 7
- Acesso pela rede e HTTPS 8

Guia das telas 9

- Entrada 9
- Biblioteca 10
- Prévia ao passar o mouse 10
- Envio de documentos 11
- Busca 12
- Detalhe do documento 13
- Leitor de PDF 14
- Leitor de DOCX 15
- Estantes 16
- Coleções 17
- Compartilhados 18
- Sistema 19

Minha conta 20
Tema escuro e celular 20

Descrição por IA 21

Provedores 21
O que é enviado ao provedor 21
Quando o provedor falha 22

Arquitetura 23

O caminho de um documento 23
Filas, retentativas e recuperação 24
Segurança 24
Estrutura de pastas 25

API REST 26

Autenticação 26
Rotas principais 26
Exemplos 27
Erros 27

Configuração 28

Aplicação 28
Banco, Redis e arquivos 28
Pipeline e IA 28
Processador Java 29
Checklist para produção 29

Testes e automação 30

Pytest 30
Integração contínua 31
Atividade diária 31

Solução de problemas 32

CAPÍTULO 1

Visão geral

O Lume é uma plataforma web para organizar e ler documentos PDF e DOCX. Você arrasta os arquivos para a janela e, em poucos segundos, cada um ganha capa, descrição escrita por IA, sumário e um leitor próprio que guarda a página onde você parou.

Para que serve

Pense numa estante digital no estilo do Scribd, só que instalada na sua máquina ou no servidor da sua equipe: apostilas, manuais internos, relatórios, contratos, livros técnicos e atas ficam num só lugar, com busca no texto completo e permissões simples (cada documento é só seu ou fica visível para todos da instalação). O projeto foi feito para ser usado como está, estudado ou adaptado.

O que o Lume faz

- **Envio por arrastar e soltar:** vários arquivos de uma vez, com barra de progresso e as etapas de cada um em tempo real. Arquivos repetidos são reconhecidos pela impressão digital (SHA-256) e não ocupam espaço de novo.
- **Capas automáticas:** a capa de um PDF é a primeira página renderizada (se ela estiver em branco, a segunda ou a terceira). DOCX ganha uma capa tipográfica com a paleta do projeto.
- **Descrição por IA ao passar o mouse:** uma prévia aparece ao lado do cartão com a descrição, as etiquetas sugeridas e os botões de leitura.
- **Leitor próprio:** PDF com pdf.js (zoom, miniaturas, sumário, busca, temas Papel, Sépia e Noite, modo foco) e DOCX convertido para HTML de leitura confortável. Destaques coloridos, notas e marcadores ficam salvos por pessoa.
- **Estantes, coleções e favoritos:** Quero ler, Lendo e Lidos com arrastar e soltar; coleções com ordem própria.
- **Busca em texto completo:** título, autor, descrição, etiquetas e o conteúdo inteiro, sem diferenciar acentos, com trecho encontrado destacado e tolerância a erros de digitação no título.
- **Tempo real:** a tela se atualiza sozinha por WebSocket quando um documento muda de etapa, em qualquer aba aberta.
- **Administração:** painel com serviços, filas, falhas, provedor de IA e contas; API REST documentada com tokens pessoais.

Tecnologias

Camada	Tecnologia	Papel
Aplicação web	Python 3.13 · Django 5.2 · Django REST Framework	Páginas, API REST, autenticação, regras da biblioteca
Tempo real	Django Channels · Uvicorn (ASGI) · WebSockets	Avisa o navegador quando um documento muda
Processamento	Java 21 · Apache PDFBox 3 · Gson	Lê PDF e DOCX, gera capa, texto, sumário e HTML de leitura

Camada	Tecnologia	Papel
Interface	JavaScript (módulos ES, sem etapa de build) · pdf.js 5	Biblioteca, prévia, leitor, estantes e painéis
Banco de dados	PostgreSQL 16 · busca `pt_unaccent` · trigramas	Fonte da verdade de documentos, leituras e anotações
Filas e eventos	Redis 7 · Streams · pub/sub	Pipeline com retentativas, eventos, cache e sessões
IA	Resumo extrativo local · Claude · APIs compatíveis com OpenAI	Descrição e etiquetas de cada documento
Execução	Docker Compose	PostgreSQL, Redis, processor, web e worker
Testes	Pytest · pytest-django · pytest-asyncio	70 testes com PostgreSQL, Redis e o processador Java reais

Identidade visual



A marca é a **Faísca**: uma página com uma estrela de quatro pontas vazada, o documento que ganha descrição escrita por IA. Ela é sempre preta e branca (preta no tema claro, branca no escuro).

A interface usa a fonte Public Sans e, nos títulos e no leitor de DOCX, a serifada Literata. As cores vêm de uma paleta de cinco tons claros; como ela não tem um tom escuro, o texto e os botões usam tons mais fechados derivados da Ardósia e da Sálvia.

Paleta

Creme
#F4F1DE
Fundo da interface e do papel.

Camurça
#B0A08C
Bordas, rótulos e detalhes neutros.

Sálvia
#B8C9A9
Descrição por IA, destaques e “Lidos”.

Ardósia
#8395A7
Progresso de leitura e estante “Lendo”.

Rosé
#D8A7B1
Favoritos, “Quero ler” e alertas suaves.

Tons derivados

Tinta
#2B3540
Texto principal.

Ardósia escura
#4A5A6B
Botões e links.

Sálvia escura
#5E7350
Rótulo da IA e sucesso.

Rosé escuro
#A65A6A
Erros e ações destrutivas.

CAPÍTULO 2

Como rodar

Com Docker, o Lume sobe com um comando. Os scripts criam o arquivo de configuração com senhas aleatórias, constroem as imagens e abrem o navegador quando tudo estiver pronto.

Requisitos

- **Docker** com Docker Compose (Docker Desktop no Windows e no macOS).
- Cerca de 2 GB de memória livre e 2 GB de disco para as imagens, mais o espaço dos seus documentos.
- Na primeira vez, acesso à internet para baixar as imagens base e as dependências do Python e do Maven.

Primeira execução

Windows

Dê dois cliques em `iniciar.cmd`, na pasta do projeto.

Linux ou macOS

```
./iniciar.sh
```

Os dois scripts fazem a mesma coisa:

1. Conferem se o Docker está instalado.
2. Na primeira vez, copiam `.env.example` para `.env` e trocam `DB_PASSWORD` e `APP_SECRET_KEY` por valores aleatórios.
3. Rodam `docker compose up -d --build`: sobem PostgreSQL, Redis, o processador Java, o servidor web e o worker.
4. Esperam a rota `/api/health` responder e abrem **http://localhost:8080**.

Endereço	O que é
http://localhost:8080	Aplicação (biblioteca, leitor, administração)
http://localhost:8080/api/docs	Documentação interativa da API (Swagger)
http://localhost:8080/api/health	Saúde do banco e do Redis (usada pelo Docker)

Para subir sem o script:

```
cp .env.example .env      # troque DB_PASSWORD e APP_SECRET_KEY
docker compose up -d --build
```

Contas de demonstração

Num banco vazio, o Lume cria duas contas e importa os 12 documentos da pasta `exemplos/` pelo mesmo pipeline de um envio normal. Os nomes, os e-mails (domínio de exemplo `lume.dev`) e os documentos são fictícios e existem só para testar.

Papel	E-mail	Senha	O que pode fazer
Administrador	`admin@lume.dev`	`Lume@2026`	Tudo, inclusive a tela Sistema e as contas
Membro	`leitor@lume.dev`	`Lume@2026`	Sua biblioteca e os documentos compartilhados

Comandos do dia a dia

Para	Comando
Ver os logs	`docker compose logs -f web worker processor`
Parar	`docker compose down` (os dados ficam nos volumes)
Atualizar depois de baixar uma versão nova	`docker compose up -d --build`
Mais workers processando ao mesmo tempo	`docker compose up -d --scale worker=3`
Abrir o terminal do Django	`docker compose exec web python manage.py shell`
Apagar tudo, inclusive os documentos	`docker compose down -v`

Usar de verdade

1. No `.env`, defina `APP_SEED_DEMO=false` antes da primeira subida (ou apague os volumes com `docker compose down -v`).
2. Se não quiser cadastro aberto, defina `APP_SIGNUP=false`.
3. Suba com `docker compose up -d --build`.
4. Crie o administrador (a senha é pedida no terminal):

```
docker compose exec web python manage.py createuser --admin \
  --email voce@suaempresa.com --name "Seu Nome"
```

As demais contas podem ser criadas pela tela **Sistema** ou, com o cadastro aberto, pela própria tela de entrada.

Rodar sem Docker

Útil para desenvolver. Você precisa de Python 3.12 ou mais novo, Java 21 com Maven, PostgreSQL 16 e Redis 7.

Processador Java

```
cd processor
mvn -B package
PROCESSOR_DATA_ROOT=../.data java -jar target/processor.jar
```

Aplicação web e worker

```
cd web
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
export DB_NAME=lume DB_USER=lume DB_PASSWORD=... DATA_ROOT=../.data \
       PROCESSOR_URL=http://localhost:8090 APP_DEBUG=true
python manage.py migrate
python manage.py seed_demo          # opcional
python manage.py worker &           # pipeline
uvicorn config.asgi:application --port 8000
```

MESMA PASTA DE DADOS

O web, o worker e o processador precisam enxergar a mesma pasta: `DATA_ROOT` (Python) e `PROCESSOR_DATA_ROOT` (Java) apontam para o mesmo lugar. No Docker isso é o volume `documents`, montado em `/data`.

Acesso pela rede e HTTPS

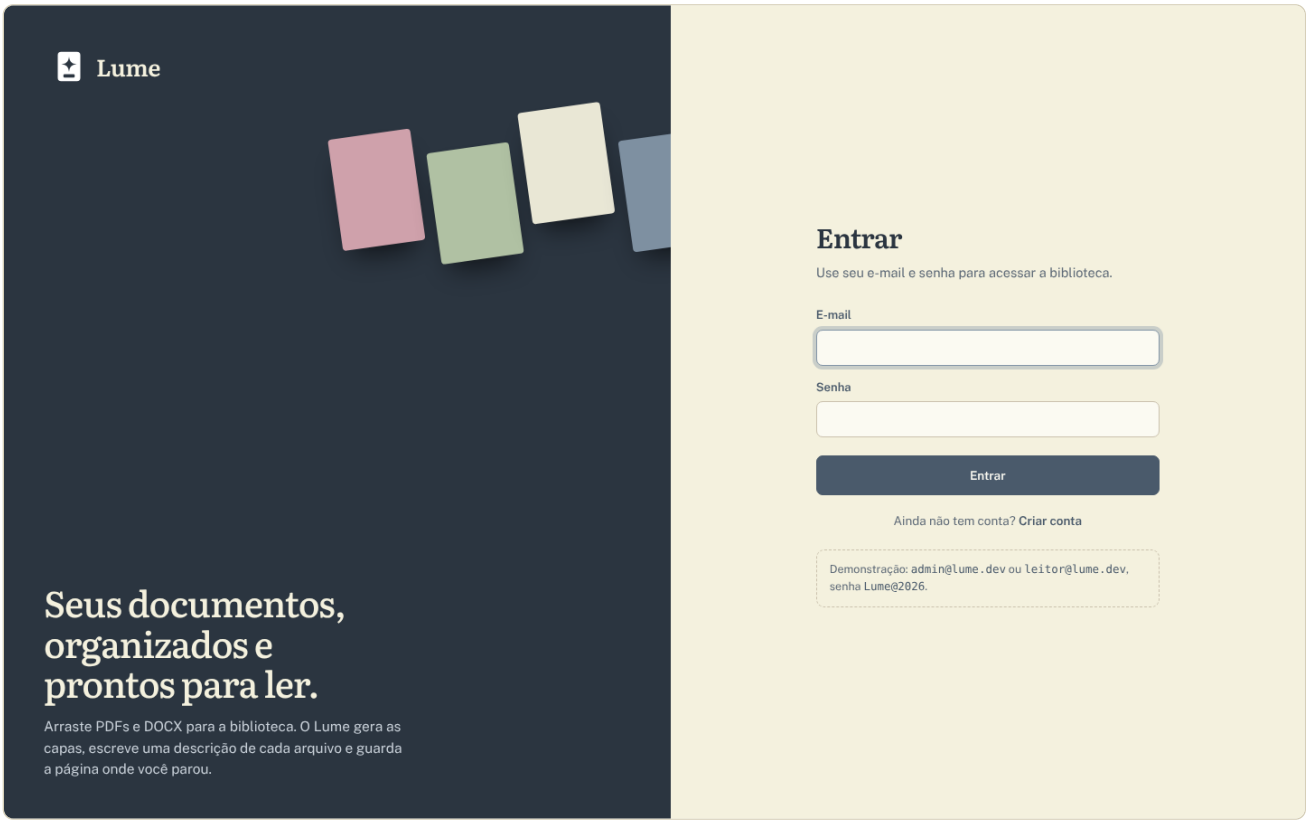
Variável	Quando mexer
`APP_ALLOWED_HOSTS`	Acrescente o IP ou o domínio usado para abrir o Lume (ex.: `192.168.0.20,lume.suaempresa.com`). Sem isso, o Django recusa a requisição e o WebSocket não conecta.
`APP_HTTPS`	`true` quando houver um proxy reverso com HTTPS na frente: os cookies passam a ser seguros e o HSTS é ativado.
`APP_CSRF_TRUSTED_ORIGINS`	Com HTTPS, a origem completa: `https://lume.suaempresa.com`.
`APP_PORT`	Porta publicada no computador (padrão 8080).

CAPÍTULO 3

Guia das telas

Cada tela, o que ela mostra e como usar. Todas funcionam no tema claro e no escuro e se adaptam ao celular.

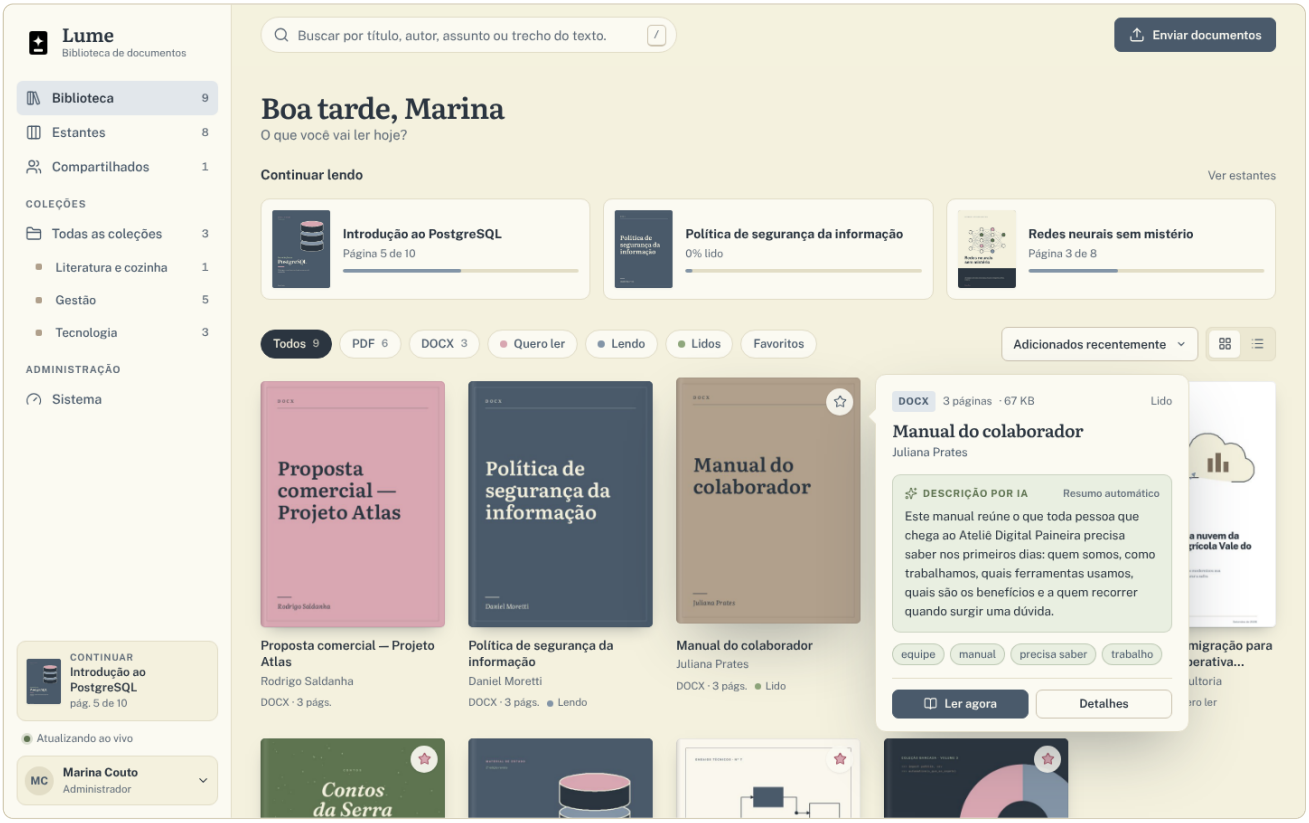
Entrada



Tela de entrada com o atalho para criar conta e o aviso das contas de demonstração.

Entre com e-mail e senha. Se o cadastro estiver aberto (`APP_SIGNUP=true`), o link **Criar conta** troca o formulário. Depois de cinco senhas erradas para o mesmo e-mail e IP, a entrada fica bloqueada por 15 minutos. O aviso de demonstração só aparece com `APP_SEED_DEMO=true`.

Biblioteca



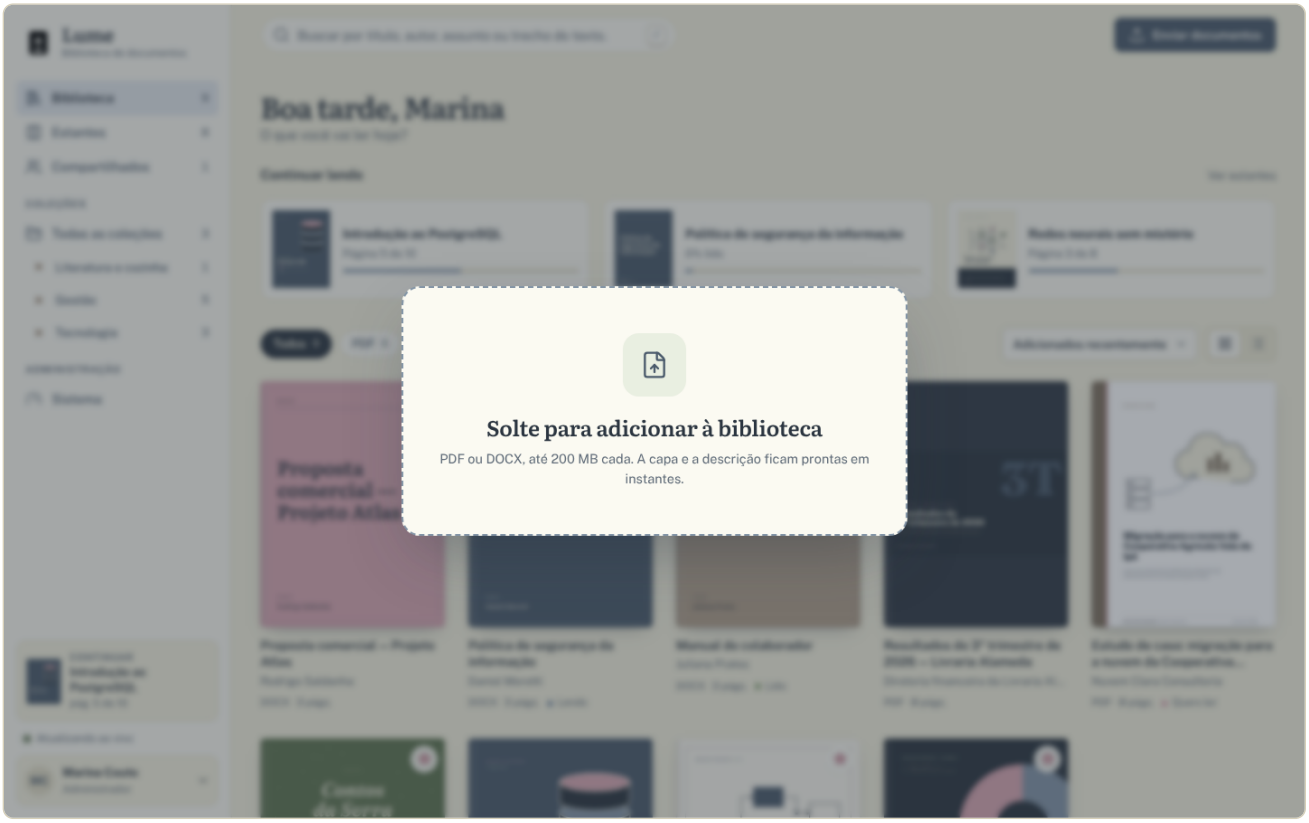
Biblioteca: continuar lendo, filtros, ordenação e a prévia aberta ao passar o mouse.

- **Continuar lendo:** os três últimos documentos abertos, com a página e a barra de progresso. Um clique abre o leitor no ponto exato.
- **Filtros:** Todos, PDF, DOCX, Quero ler, Lendo, Lidos e Favoritos. Quando existem, aparecem também Processando e Com erro.
- **Ordenação:** adicionados recentemente, lidos recentemente, título, autor, mais páginas ou maior tamanho (e “mais relevantes” durante uma busca).
- **Grade ou lista:** o botão ao lado da ordenação alterna; a escolha fica lembrada no navegador.
- **Cartão:** capa, título, autor, tipo e páginas, estante e progresso. A estrela marca o favorito. Arraste um cartão até uma coleção no menu lateral para incluí-lo nela.
- A lista carrega mais documentos sozinha ao rolar a página.

Prévia ao passar o mouse

Pare o mouse sobre uma capa por um instante (ou navegue com Tab): a prévia abre ao lado, com tipo, páginas, tamanho, a **descrição por IA** (e quem a escreveu: resumo automático, Claude ou o modelo configurado), as etiquetas e os botões **Ler agora** (ou **Continuar**) e **Detalhes**. Enquanto o documento é processado, a prévia avisa que a descrição ainda vai chegar.

Envio de documentos



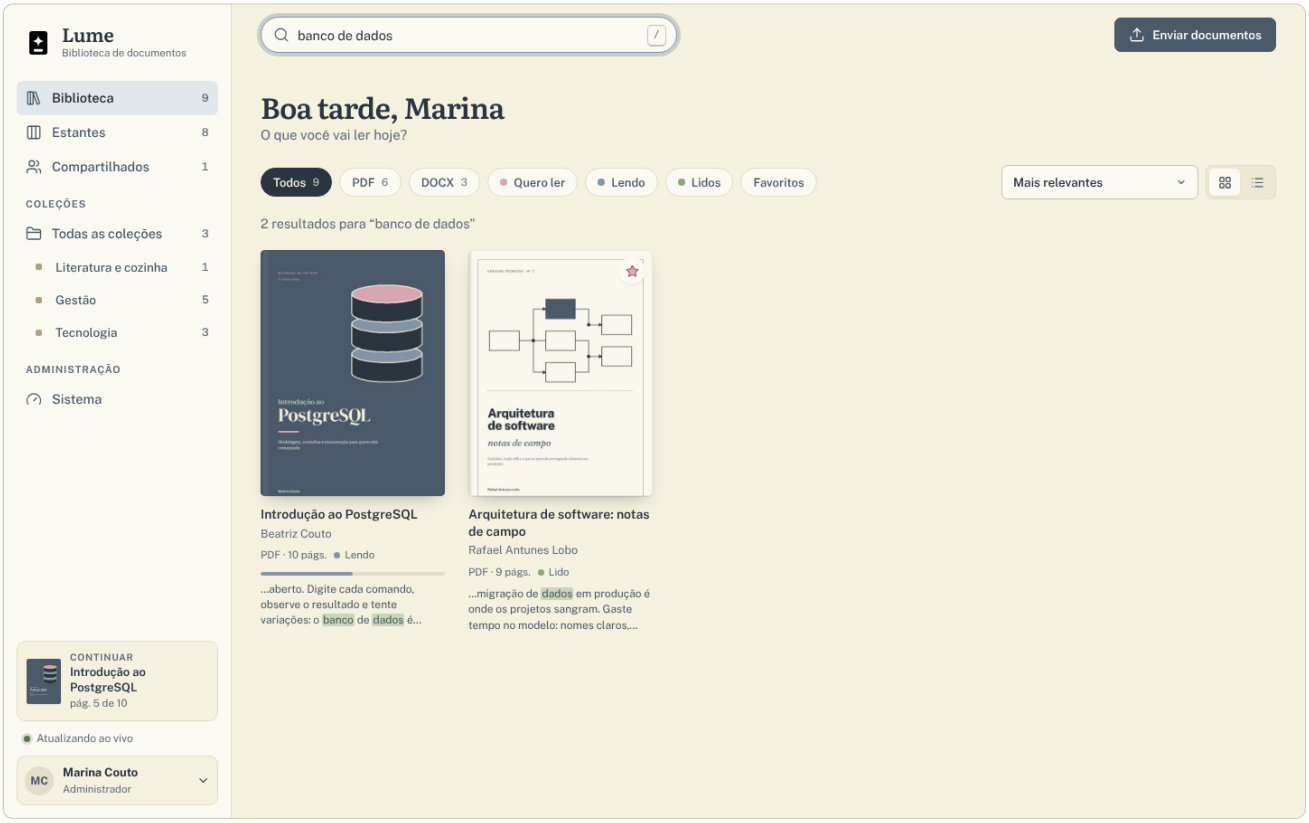
Arraste arquivos para qualquer lugar da janela.

1. Arraste PDFs e DOCX para a janela, clique em **Enviar documentos** ou aperte U.
2. A bandeja no canto mostra cada arquivo: *Enviando* (com porcentagem), *Na fila*, *Lendo o documento*, *Escrevendo a descrição* e *Pronto*.
3. O cartão aparece na biblioteca na hora, com um esqueleto animado no lugar da capa, que surge quando o processador termina.
4. Se o envio for feito dentro de uma coleção, o documento já entra nela.

LIMITES

Até 50 arquivos por vez e 200 MB por arquivo (`APP_MAX_UPLOAD_MB`). O tipo é conferido pelo conteúdo, não pela extensão: um .txt renomeado para .pdf é recusado. Arquivos .doc do Word 97–2003 também: abra no Word e salve como .docx.

Busca



Busca por “banco de dados”: os trechos encontrados aparecem marcados nos cartões.

A caixa no topo busca em título, autor, descrição, etiquetas e no texto inteiro dos documentos. Acentos e maiúsculas não importam (“decisoes tecnicas” encontra “decisões técnicas”), palavras derivadas também (“tabelas” encontra “tabela”) e pequenos erros no título são tolerados. Aceita aspas para frase exata e `-palavra` para excluir. Atalho: `/`.

Detalhe do documento

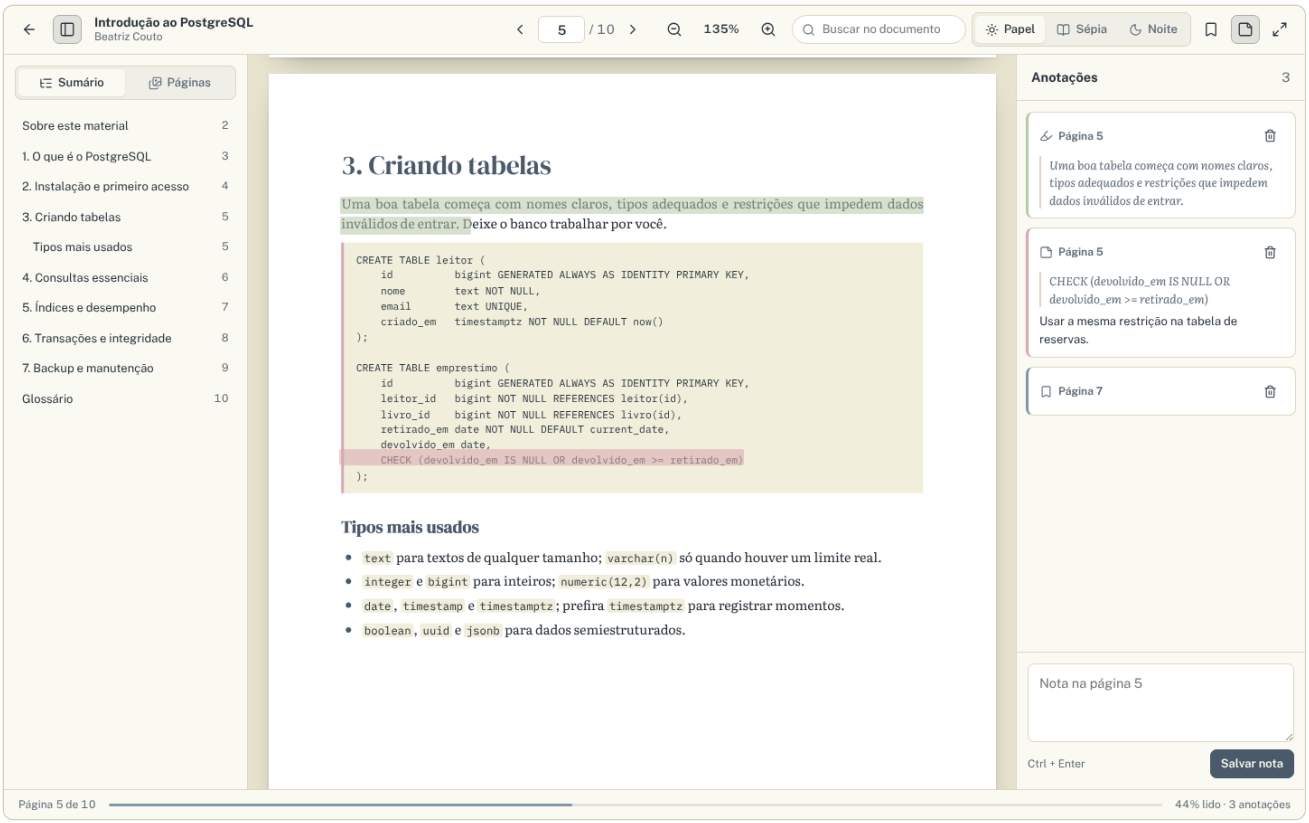


Detalhe: capa, ações, descrição por IA, etiquetas, leitura, coleções e sumário.

Ação	Onde	O que faz
Ler agora / Continuar lendo	Botão sob a capa	Abre o leitor na página salva
Baixar	Ícone de download	Baixa o arquivo original
Editar informações	Menu ⋯	Título, autor e sua descrição. Depois disso, a leitura do arquivo não sobrescreve mais título e autor.
Compartilhar com todos / Só para mim	Menu ⋯	Torna o documento visível (só leitura) para todas as contas
Ler o arquivo de novo	Menu ⋯	Reenvia para o processador (útil depois de atualizar o Lume)
Gerar de novo	Bloco da descrição	Pede outra descrição, por exemplo depois de trocar o provedor de IA
Etiquetas	Editar	Suas etiquetas substituem as sugeridas pela IA
Minha leitura	Quero ler · Lendo · Lido · Favoritar	Move entre estantes e marca o favorito
Coleções	Chips	Inclui ou tira o documento de cada coleção; cria uma nova
Sumário	Lista	Cada item abre o leitor direto naquela parte
Excluir	Menu ⋯	Apaga o documento, os arquivos, as anotações e o progresso

Documentos de outras pessoas (compartilhados) mostram quem enviou e não podem ser editados; anotações e progresso de leitura continuam sendo seus.

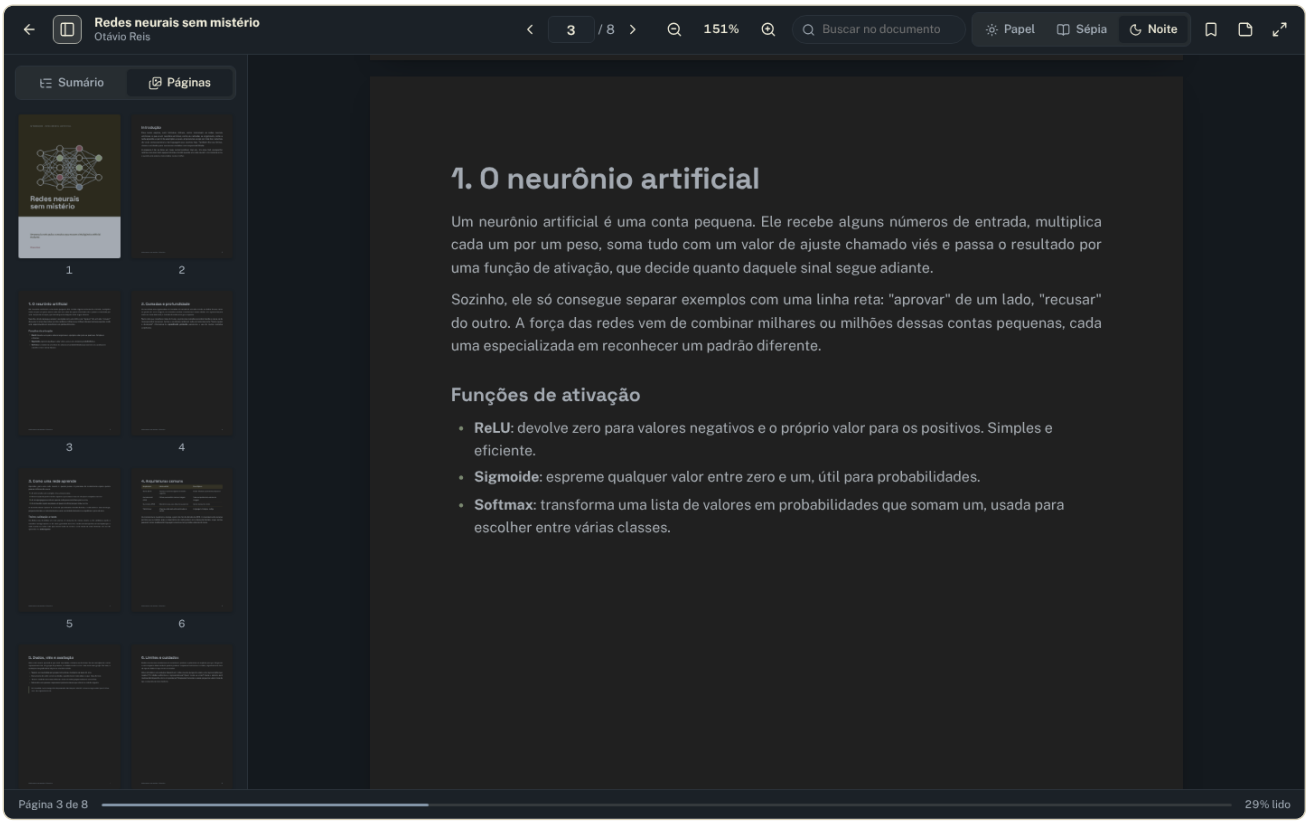
Leitor de PDF



Leitor de PDF com sumário, destaques, nota e marcador no painel de anotações.

Recurso	Como usar
Páginas	Rolagem contínua; campo de página, setas ← →, Page Up Page Down, Home End
Zoom	Botões − e +, teclas + e −; clique na porcentagem para voltar a ajustar à largura
Sumário e miniaturas	Botão do painel esquerdo; abas Sumário e Páginas
Busca no documento	Caixa “Buscar no documento” ou Ctrl+F; Enter vai ao próximo, Shift+Enter ao anterior
Temas da página	Papel, Sépia e Noite (a escolha fica salva na sua conta)
Destaques	Selecione um trecho: escolha uma das quatro cores (Sálvia, Rosé, Ardósia, Camurça), anote ou copie
Notas e marcadores	Painel direito: nota na página atual (Ctrl+Enter salva), ícone de marcador no topo
Modo foco	Ícone de tela cheia ou F; Esc sai do foco e, de novo, do leitor
Progresso	Salvo sozinho ao trocar de página e ao sair; o documento entra em “Lendo”

O PDF é baixado em partes (requisições *Range*): arquivos grandes começam a aparecer antes do download terminar, e só as páginas próximas da tela ficam renderizadas na memória.



Tema Noite com o painel de miniaturas.

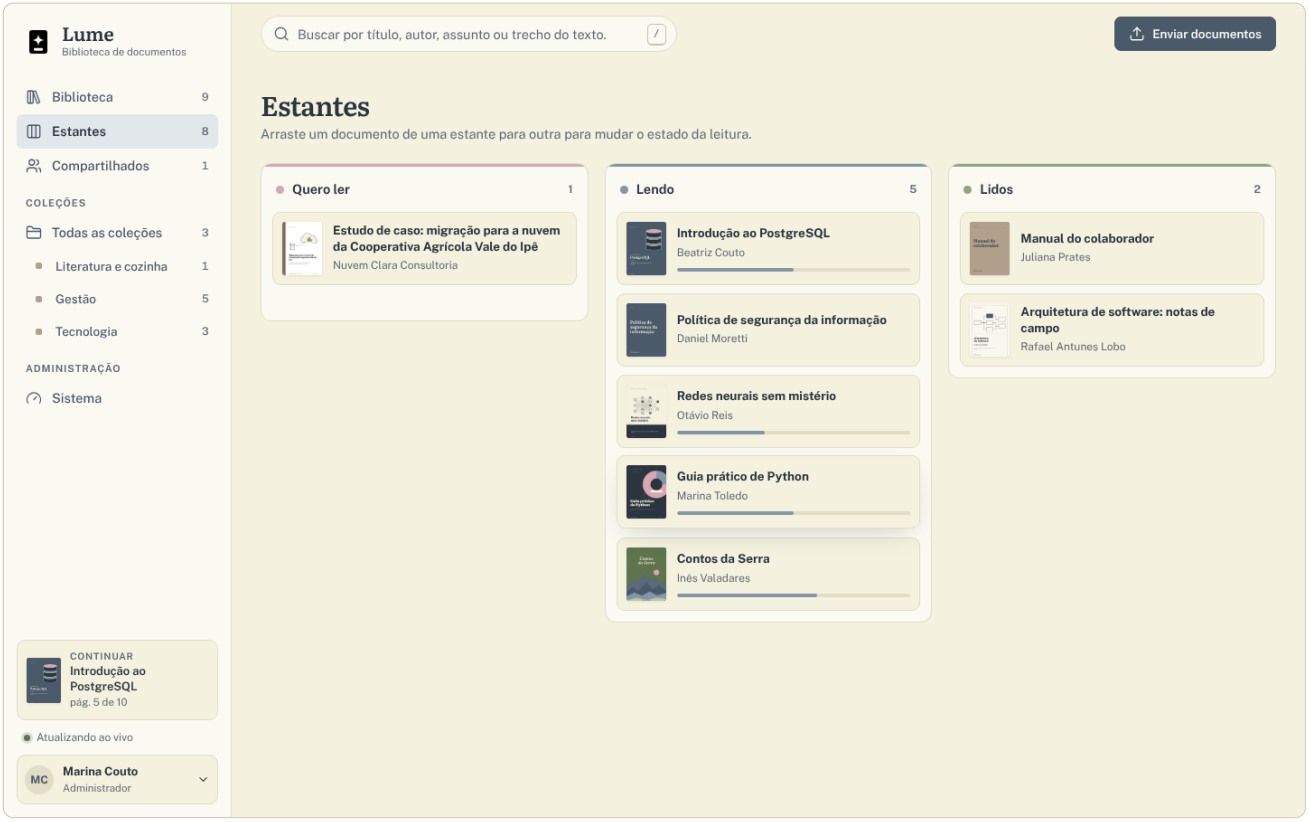
Leitor de DOCX



DOCX convertido em HTML de leitura, no tema Sépia, com o sumário dos títulos.

O DOCX é convertido pelo processador Java em HTML limpo: títulos, listas, tabelas, citações, links e imagens. Ajuste o tamanho da letra com os botões A− e A+ (ou + e −). O progresso é salvo pela posição do parágrafo; destaques, notas, marcadores, busca e temas funcionam como no PDF.

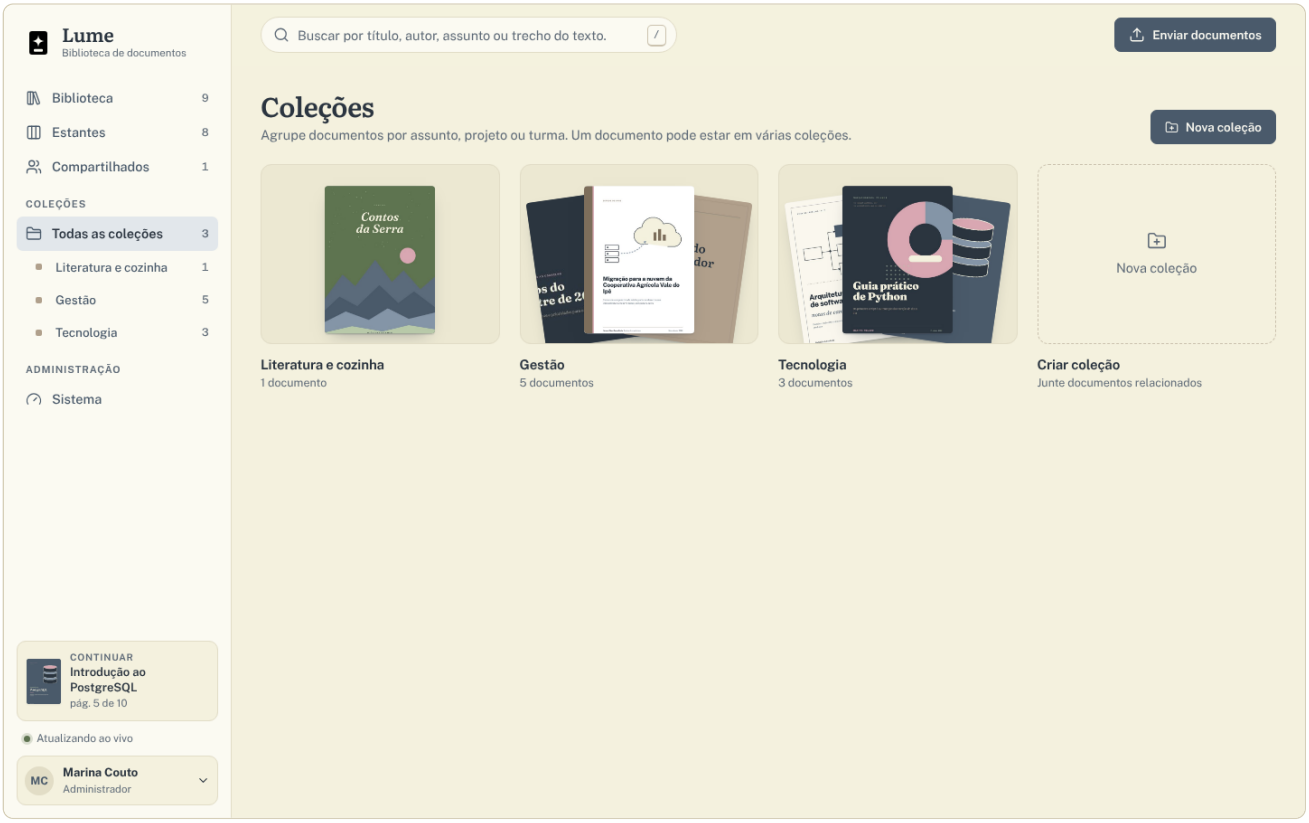
Estantes



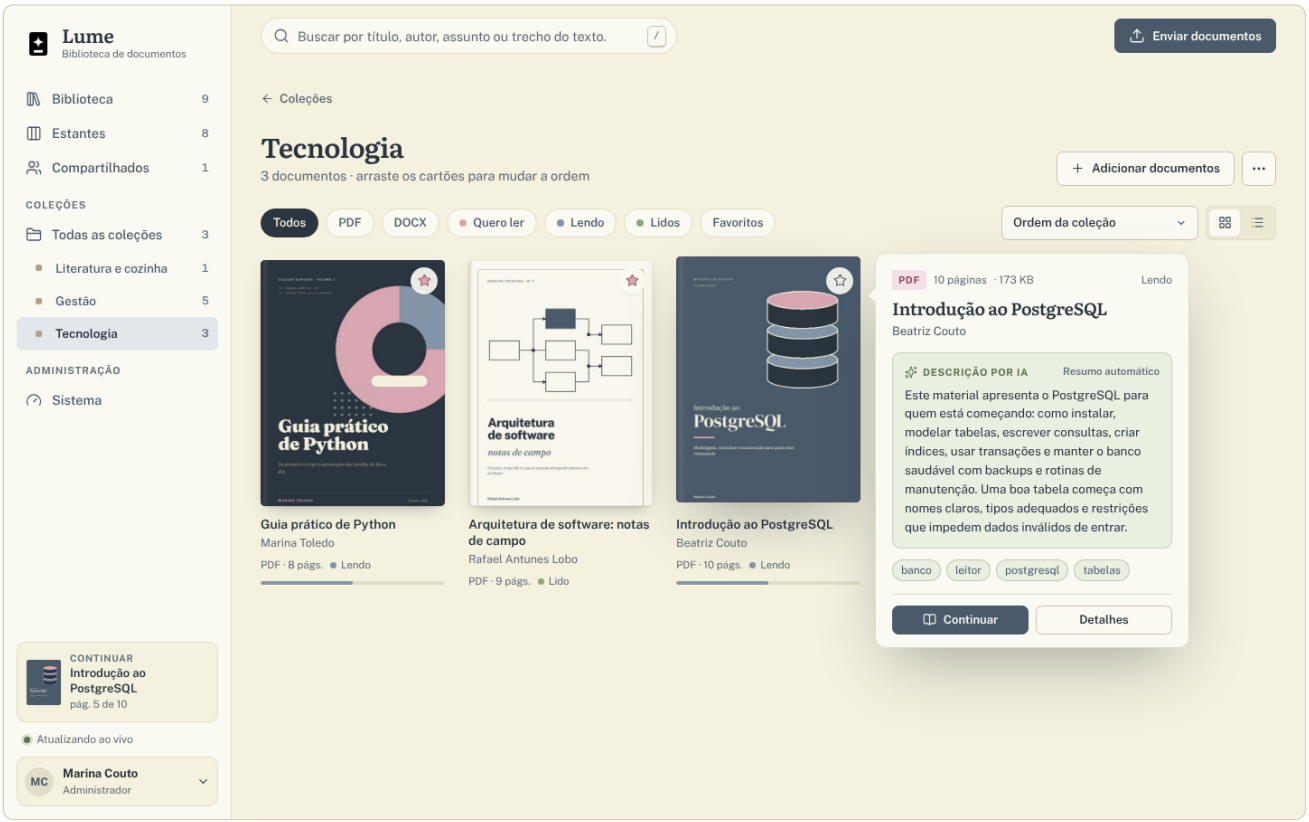
Estantes: arraste um documento de uma coluna para outra.

Três colunas: **Quero ler**, **Lendo** e **Lidos**. Arraste os cartões entre elas. Abrir um documento no leitor já o coloca em Lendo; marcar como lido registra a data de conclusão.

Coleções



Coleções com as capas empilhadas.



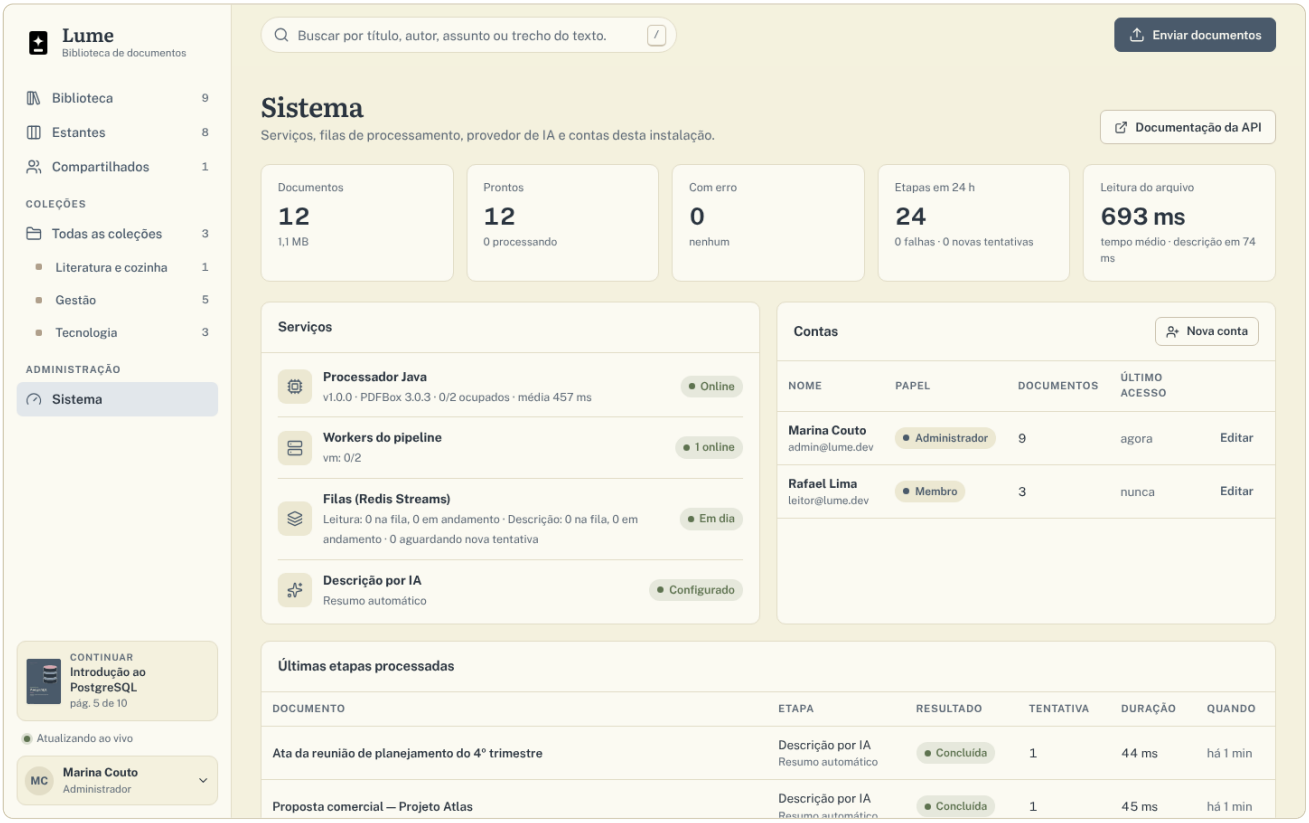
Dentro de uma coleção: adicione documentos e arraste os cartões para mudar a ordem.

- **Nova coleção:** botão no topo ou no próprio documento.
- **Adicionar documentos:** dentro da coleção, escolha numa lista; ou arraste o cartão para o nome da coleção no menu lateral.
- **Reordenar:** na ordem “Ordem da coleção”, arraste os cartões.
- **Renomear e excluir:** menu ⋯ da coleção. Excluir a coleção não apaga os documentos.

Compartilhados

Mostra os documentos que outras pessoas deixaram visíveis para todos, com o nome de quem enviou. Você pode ler, anotar, favoritar e colocar nas suas estantes e coleções.

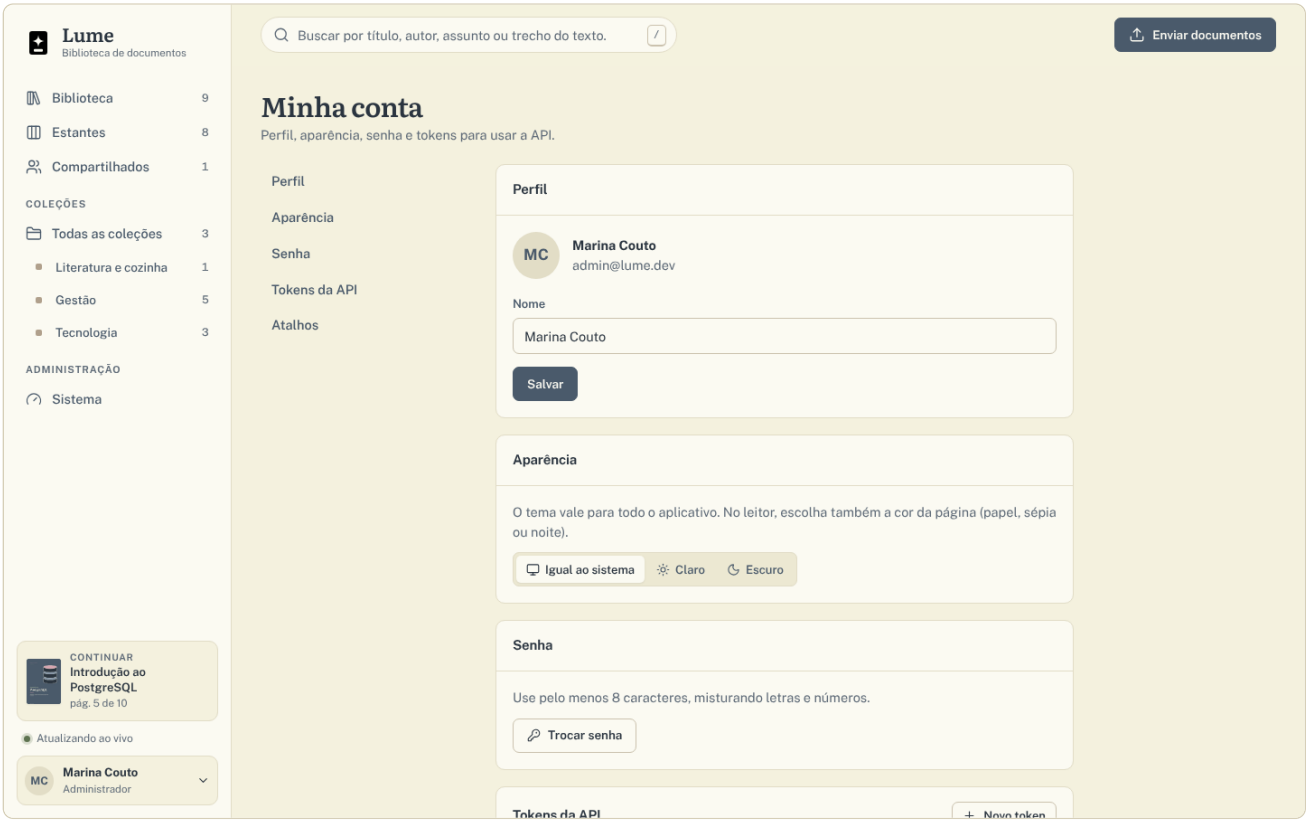
Sistema



Sistema (só administradores): números, serviços, contas e últimas etapas processadas.

- **Números:** documentos e espaço usado, prontos, com erro, etapas nas últimas 24 h e tempo médio de leitura e de descrição.
- **Serviços:** processador Java (versão do PDFBox, vagas ocupadas, tempo médio), workers online, filas do Redis e provedor de IA.
- **Documentos com erro:** o motivo de cada falha e o botão *Tentar todos de novo*.
- **Contas:** criar, editar papel, desativar e trocar senha.
- **Últimas etapas:** cada tentativa de leitura e de descrição, com duração e erro. A tela se atualiza sozinha.

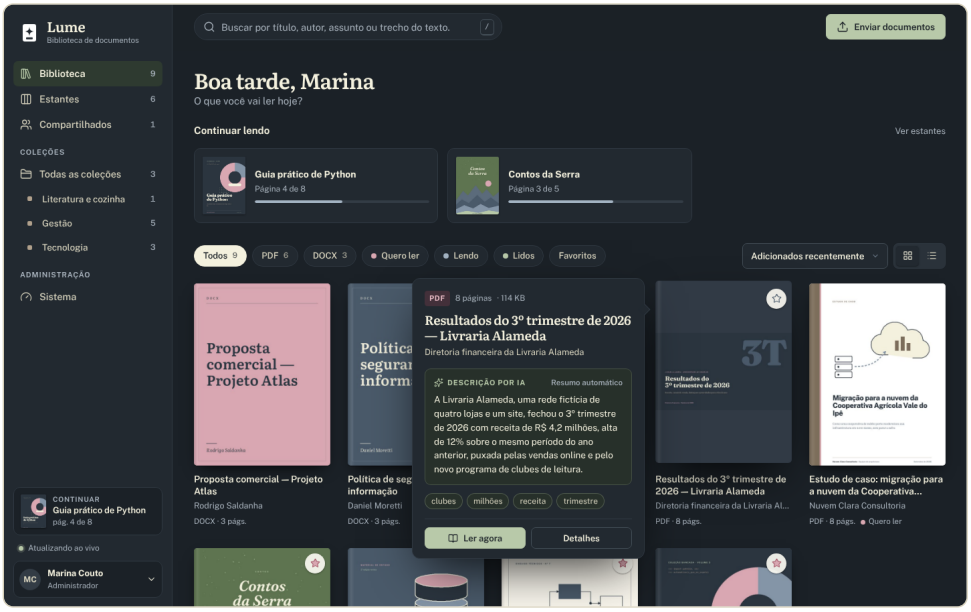
Minha conta



Perfil, aparência, senha, tokens da API e atalhos de teclado.

Troque nome, tema (igual ao sistema, claro ou escuro) e senha. Em **Tokens da API**, crie um token para scripts: o valor aparece uma única vez e só o hash fica guardado. Cada conta pode ter até 20 tokens ativos.

Tema escuro e celular



Tema escuro: segue o sistema ou a escolha feita em Minha conta.



No celular, o menu vira gaveta.

CAPÍTULO 4

Descrição por IA

Cada documento recebe uma descrição de duas ou três frases e até quatro etiquetas. O provedor é escolhido no .env e pode ser trocado a qualquer momento.

Provedores

AI_PROVIDER	Como funciona	O que configurar
`local` (padrão)	Resumo extrativo feito no próprio worker: escolhe as frases mais representativas (frequência de termos, peso do título e dos títulos de seção, posição, corte de redundância) e extrai palavras-chave. Não envia nada para fora e não precisa de chave.	Nada.
`anthropic`	API de Mensagens da Anthropic (Claude).	`AI_API_KEY`; `AI_MODEL` é opcional (padrão `claude-haiku-5-5`).
`openai`	Qualquer API compatível com `/v1/chat/completions`: OpenAI, Ollama, LM Studio, OpenRouter e outras.	`AI_MODEL` (obrigatório), `AI_BASE_URL` (padrão da OpenAI) e `AI_API_KEY` quando o serviço pedir.

Exemplo com Claude

```
AI_PROVIDER=anthropic
AI_API_KEY=sua-chave
AI_MODEL=claude-haiku-5-5
```

Exemplo com um modelo local (Ollama)

```
docker compose --profile ia-local up -d
docker compose exec ollama ollama pull llama3.2

# no .env
AI_PROVIDER=openai
AI_BASE_URL=http://ollama:11434/v1
AI_MODEL=llama3.2
```

Depois de mudar o `.env`, aplique com `docker compose up -d`. Os documentos já descritos continuam como estão; use **Gerar de novo** no detalhe do documento para refazer.

O que é enviado ao provedor

Só o necessário para descrever: título, autor, tipo, número de páginas, até 30 títulos do sumário e um trecho do texto já limpo (sem capa, cabeçalhos e rodapés repetidos) de até `AI_MAX_INPUT_CHARS` caracteres (padrão 12 000): três quartos do início e um quarto do meio do documento. O pedido exige resposta em JSON com a descrição em português e as etiquetas.

DOCUMENTOS CONFIDENCIAIS

Com `anthropic` ou `openai`, esse trecho sai do seu servidor. Para documentos sigilosos, mantenha o provedor `local` ou use um modelo rodando na sua rede (Ollama).

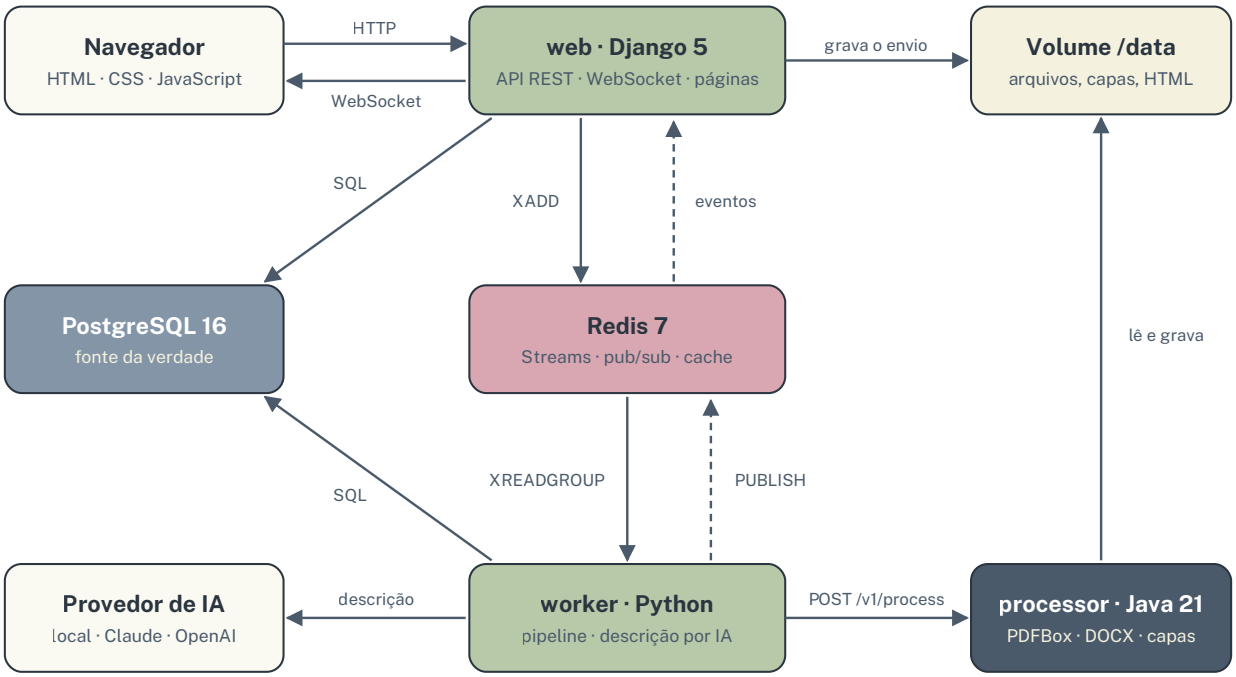
Quando o provedor falha

Se a API externa demorar mais que `AI_TIMEOUT` (60 s), responder com erro, vier sem JSON ou com uma descrição vazia, o worker usa o resumo local na hora: o documento fica pronto do mesmo jeito e o motivo aparece em **Sistema → Últimas etapas**. Documentos sem texto (PDF digitalizado como imagem) recebem um aviso no lugar da descrição. As etiquetas da IA nunca apagam as que você escolheu.

CAPÍTULO 5

Arquitetura

Cinco serviços no Docker Compose. O PostgreSQL guarda o estado; o Redis distribui o trabalho e os avisos; o processador Java não tem acesso ao banco e só lê e grava arquivos.



Serviço	Comando	Responsabilidade
postgres	imagem oficial 16	Documentos, contas, leituras, anotações, coleções e histórico do processamento
redis	imagem oficial 7, AOF ligado	Filas (Streams), retentativas, eventos (pub/sub), cache, sessões e bloqueio de login
processor	`java -jar processor.jar`	API HTTP interna (porta 8090): lê PDF e DOCX, gera capa, texto, sumário e HTML
web	`./start.sh` (migrações + Uvicorn)	Páginas, API REST, WebSocket, arquivos estáticos e envio
worker	`python manage.py worker`	Consome as filas, chama o processador, escreve a descrição e recupera falhas

O caminho de um documento

1. **Envio.** O web confere tamanho e tipo pelo conteúdo, calcula o SHA-256 enquanto grava, recusa repetidos e salva em `/data/documents/<id>/original.pdf`. Cria o documento com estado *Na fila* e, só depois do commit, publica no stream `docs:extract`.
2. **Leitura.** Um worker assume a mensagem (`XREADGROUP`), marca *Lendo o documento* com `SELECT … FOR UPDATE SKIP LOCKED` e chama `POST /v1/process` no processador.

3. **Processador Java.** Para PDF: metadados, número de páginas, texto (páginas separadas por quebra de página), sumário pelos marcadores e capa da primeira página não vazia. Para DOCX: lê o pacote com StAX, gera `reader.html`, extrai imagens, monta o sumário pelos títulos e desenha a capa tipográfica.
4. **Gravação.** O worker higieniza o HTML de novo, guarda metadados, texto e sumário, atualiza o índice de busca e publica em `docs:describe`.
5. **Descrição.** O worker gera descrição, idioma e etiquetas, marca *Pronto* e publica o evento.
6. **Tempo real.** A cada mudança, um aviso vai para o canal `events:user:<id>` (e `events:shared` se for compartilhado); o web entrega pelo WebSocket e o navegador busca o documento atualizado.

Filas, retentativas e recuperação

Peça	Como funciona
Streams `docs:extract` e `docs:describe`	Grupo de consumidores `workers`; cada worker tem `WORKER_THREADS` linhas de execução e confirma (`XACK`) só depois de terminar.
Mensagem repetida	A etapa só começa se o documento estiver no estado certo; uma mensagem duplicada ou atrasada não faz nada.
Erro passageiro	Processador fora do ar, rede, 503: nova tentativa com espera exponencial — 5 s, 15 s, 45 s, até 300 s — guardada no ZSET `docs:delayed`. Depois de `PIPELINE_MAX_ATTEMPTS` (4), o documento fica *Falhou*.
Arquivo recusado	PDF com senha, arquivo corrompido ou formato inválido (resposta 422): falha na hora, com o motivo em português, sem gastar tentativas.
Worker que morreu	Mensagens paradas há mais de `PIPELINE_VISIBILITY_TIMEOUT` (600 s) são reassumidas com `XAUTOCLAIM`.
Redis apagado	Os grupos são recriados sozinhos e a varredura devolve à fila os documentos parados há mais de `PIPELINE_STALE_AFTER` (1 800 s). Nada se perde: o estado está no PostgreSQL.
Batimento	Cada worker grava `workers:<nome>` a cada 5 s com validade de 30 s; é o que a tela Sistema mostra.

TEMPO LIMITE

O worker espera o processador por até `PROCESSOR_TIMEOUT` (300 s), menos que o tempo de reassumir uma mensagem (600 s). Assim, um documento muito grande nunca é processado por dois workers ao mesmo tempo.

Segurança

Onde	Proteção
Navegador	Content-Security-Policy sem scripts inline, `X-Frame-Options: DENY`, nosniff, Referrer-Policy, Permissions-Policy; HSTS com `APP_HTTPS=true`.
Contas	Senhas com PBKDF2 e validadores do Django; bloqueio de 15 min após 5 erros por IP + e-mail; CSRF em todo POST com sessão; sessão de 14 dias com cookie HttpOnly.

Onde	Proteção
Tokens da API	Formato `dl_<prefixo>_<segredo>`; só o SHA-256 é guardado; podem expirar e ser revogados.
Envio	Tipo conferido pelo conteúdo, limite de tamanho por arquivo e por envio, arquivo temporário apagado em qualquer erro.
Arquivos	Servidos só pela API, com checagem de permissão; nomes de mídia validados; suporte a Range e ETag.
Processador Java	Sem acesso a banco ou Redis; PathGuard impede sair de `/data`; XML lido com StAX sem DTD nem entidades externas (XXE); limites contra zip-bomb (10 000 entradas, 64 MB por XML, 512 MB no total); PDFBox com cache em arquivo temporário; fila de capacidade com resposta 503.
HTML de leitura	Gerado com lista restrita no Java e higienizado de novo com nh3 (só tags e atributos permitidos, links http/https/mailto com `noopener`).
Busca	Trechos encontrados escapados antes de virar HTML.

Estrutura de pastas

```
Document-Library-Platform/
├── docker-compose.yml        serviços, volumes e perfil ia-local
├── .env.example              todas as configurações (sem segredos)
├── iniciar.cmd · iniciar.sh  primeira execução em um comando
├── web/                      aplicação Python
│   ├── config/               settings, rotas, ASGI
│   ├── apps/
│   │   ├── accounts/         contas, tokens, bloqueio de login
│   │   ├── library/          documentos, busca, estantes, coleções, anotações
│   │   ├── processing/       filas, worker, etapas, cliente do processador
│   │   ├── ai/               resumo local e provedores externos
│   │   ├── realtime/         WebSocket e distribuidor de eventos
│   │   └── core/             arquivos com Range, eventos, segurança
│   ├── templates/ · static/  páginas, CSS, JavaScript, fontes, pdf.js
│   └── tests/                Pytest
├── processor/                serviço Java (Maven)
│   └── src/main/java/io/github/havelroot/processor/
│       ├── http/  pdf/  docx/  cover/
├── exemplos/                 12 documentos fictícios + manifest.json
├── docs/                     imagens e cópias dos PDFs
└── .github/workflows/        ci.yml e diario.yml
```

CAPÍTULO 6

API REST

Tudo o que a interface faz está na API. A documentação interativa, gerada a partir do código, fica em /api/docs (Swagger) e o esquema OpenAPI em /api/schema.

Autenticação

- **No navegador:** a sessão da tela de entrada. Pedidos que alteram dados levam o cabeçalho `X-CSRFToken` com o valor do cookie `csrftoken`.
- **Em scripts:** crie um token em Minha conta e envie `Authorization: Bearer dl_…`. Tokens não precisam de CSRF.

Rotas principais

Método	Rota	Para quê
GET	/api/v1/documents	Lista com `q`, `scope` (mine, shared, all), `kind`, `status`, `shelf`, `favorite`, `collection`, `tag`, `sort`, `page`, `page_size`
POST	/api/v1/documents	Envio multipart: campo `files` (repetido), `visibility`, `collection`
GET · PATCH · DELETE	/api/v1/documents/{id}	Detalhe; editar título, autor, descrição e visibilidade; excluir
GET	/documents/{id}/file	Arquivo original (Range; `?download=1` força o download)
GET	/documents/{id}/cover	Capa em JPEG
GET	/documents/{id}/reader	HTML de leitura (DOCX) e sumário
POST	/documents/{id}/reprocess · /describe	Ler o arquivo de novo · gerar a descrição de novo
PUT	/documents/{id}/tags	Etiquetas do usuário
GET · PUT · PATCH	/documents/{id}/reading	Estante, página, progresso, posição e favorito
POST	/documents/{id}/open	Registra a abertura no leitor
GET · POST	/documents/{id}/annotations	Destaques, notas e marcadores (PATCH e DELETE em `/annotations/{id}`)
GET · POST	/api/v1/collections	Coleções (GET, PATCH, DELETE em `/{id}`; POST `/{id}/documents`; PUT `/{id}/order`)
GET	/api/v1/tags · /api/v1/stats	Etiquetas com contagem · números do menu lateral
GET · PATCH	/api/v1/me	Perfil, tema e preferências do leitor (senha em `/me/password`, tokens em `/me/tokens`)
GET	/api/v1/system	Administração: filas, workers, processador, IA e falhas
GET · POST	/api/v1/users	Administração: contas
WS	/ws/events	Avisos em tempo real (sessão do navegador)

Exemplos

Enviar dois arquivos

```
curl -H "Authorization: Bearer $TOKEN" \
  -F files=@relatorio.pdf -F files=@ata.docx \
  http://localhost:8080/api/v1/documents
```

Buscar

```
curl -H "Authorization: Bearer $TOKEN" \
  "http://localhost:8080/api/v1/documents?q=banco+de+dados&scope=all"
```

Baixar o original

```
curl -H "Authorization: Bearer $TOKEN" -OJ \
  "http://localhost:8080/api/v1/documents/$ID/file?download=1"
```

Erros

Os erros voltam sempre no mesmo formato, com a mensagem em português:

```
{"error": "invalid_credentials", "message": "E-mail ou senha incorretos."}
```

Código	Quando
400	Dados inválidos (com `fields` indicando cada campo) ou arquivo recusado
401 · 403	Sem sessão ou token · sem permissão (ex.: editar documento de outra pessoa)
404	Não existe ou não é visível para você
409	Conflito (ex.: gerar descrição de um documento ainda em processamento)
429	Muitas tentativas de entrada

CAPÍTULO 7

Configuração

Tudo é configurado por variáveis de ambiente no arquivo .env. Os valores abaixo são os padrões quando a variável fica vazia.

Aplicação

Variável	Padrão	Para que serve
`APP_PORT`	8080	Porta do navegador
`APP_SECRET_KEY`	—	Chave secreta do Django (gerada pelo script)
`APP_ALLOWED_HOSTS`	localhost,127.0.0.1	Endereços aceitos
`APP_HTTPS`	false	Cookies seguros e HSTS atrás de proxy HTTPS
`APP_CSRF_TRUSTED_ORIGINS`	—	Origens confiáveis com HTTPS
`APP_SEED_DEMO`	true	Contas e documentos de demonstração num banco vazio
`APP_SIGNUP`	true	Cadastro aberto na tela de entrada
`APP_MAX_UPLOAD_MB`	200	Tamanho máximo por arquivo
`APP_MAX_FILES_PER_UPLOAD`	50	Arquivos por envio
`APP_TIMEZONE`	America/Sao_Paulo	Fuso das datas
`APP_LOG_LEVEL`	INFO	Detalhe dos logs
`APP_DEBUG`	false	Só para desenvolvimento
`WEB_WORKERS`	2	Processos do Uvicorn

Banco, Redis e arquivos

Variável	Padrão	Para que serve
`DB_NAME` · `DB_USER`	lume	Banco e usuário do PostgreSQL
`DB_PASSWORD`	—	Senha do PostgreSQL (gerada pelo script)
`DB_HOST` · `DB_PORT`	postgres · 5432	Endereço (a porta também é publicada só em 127.0.0.1)
`REDIS_URL`	redis://redis:6379/0	Redis
`DATA_ROOT`	/data	Pasta dos documentos (volume compartilhado)

Pipeline e IA

Variável	Padrão	Para que serve
`WORKER_THREADS`	2	Documentos processados ao mesmo tempo por worker

Variável	Padrão	Para que serve
`PROCESSOR_URL`	http://processor:8090	Endereço do processador
`PROCESSOR_TIMEOUT`	300	Espera máxima por documento (s)
`PIPELINE_MAX_ATTEMPTS`	4	Tentativas antes de marcar Falhou
`PIPELINE_VISIBILITY_TIMEOUT`	600	Segundos até reassumir mensagem de worker morto
`PIPELINE_STALE_AFTER`	1800	Segundos até a varredura devolver documento parado
`AI_PROVIDER`	local	local, anthropic ou openai
`AI_API_KEY`	—	Chave do provedor externo
`AI_MODEL`	claude-haiku-5-5 (anthropic)	Modelo
`AI_BASE_URL`	da API oficial	Outro endereço compatível (Ollama, LM Studio…)
`AI_TIMEOUT`	60	Espera pela resposta da IA (s)
`AI_MAX_INPUT_CHARS`	12000	Tamanho do trecho enviado

Processador Java

Variável	Padrão	Para que serve
`PROCESSOR_PORT`	8090	Porta HTTP interna
`PROCESSOR_DATA_ROOT`	/data	Raiz dos arquivos (nada fora dela é lido)
`PROCESSOR_CONCURRENCY`	2	Documentos lidos ao mesmo tempo
`PROCESSOR_ACQUIRE_TIMEOUT`	60	Espera por vaga antes de responder 503 (s)
`PROCESSOR_MAX_TEXT_CHARS`	2000000	Limite do texto extraído
`PROCESSOR_LOG_LEVEL`	INFO	DEBUG, INFO, WARN ou ERROR

Checklist para produção

- Senha do banco e `APP_SECRET_KEY` longas e aleatórias (os scripts já geram); o `.env` nunca vai para o Git.
- `APP_SEED_DEMO=false` e, se for o caso, `APP_SIGNUP=false`.
- HTTPS num proxy reverso (Nginx, Caddy, Traefik) com `APP_HTTPS=true`, `APP_ALLOWED_HOSTS` e `APP_CSRF_TRUSTED_ORIGINS`; o proxy precisa repassar o WebSocket em `/ws/`.
- Backup diário do PostgreSQL (`pg_dump`) e do volume `documents` — os dois juntos.
- Workers e `PROCESSOR_CONCURRENCY` conforme a CPU; acompanhe a tela Sistema.
- Para IA externa com documentos sensíveis, prefira um modelo na sua rede.

CAPÍTULO 8

Testes e automação

Os testes rodam contra PostgreSQL e Redis de verdade, e os testes de ponta a ponta usam o processador Java compilado.

Pytest

```
cd web
pip install -r requirements-dev.txt
export DB_NAME=lume DB_USER=lume DB_PASSWORD=...
python -m pytest
```

São 70 testes, com cobertura por volta de 90%:

Arquivo	O que cobre
`test_accounts.py`	Entrada, bloqueio após 5 erros, CSRF, cadastro, senha, tokens e administração de contas
`test_upload.py`	Envio, detecção pelo conteúdo, repetidos, arquivos inválidos, limite de tamanho, coleção e visibilidade
`test_documents.py`	Privacidade, compartilhamento, edição, busca sem acento e com trechos, filtros, leitura, anotações, coleções, Range, exclusão
`test_pipeline.py`	Etapas, mensagens repetidas, recusa, retentativas com espera, higienização do HTML, recuperação e Redis apagado
`test_ai.py`	Resumo local, idioma, etiquetas, Claude e APIs compatíveis simulados, fallback e chave nunca exposta
`test_realtime.py`	WebSocket: recusa sem login, eventos próprios e compartilhados
`test_system.py`	Saúde, tela Sistema, esquema OpenAPI
`test_e2e_processor.py`	PDF, DOCX e PDF corrompido passando pelo processador Java real

ISOLAMENTO

Os testes usam um banco próprio criado pelo pytest-django e o banco 15 do Redis (mude com `TEST_REDIS_URL`), que é limpo a cada teste. O banco do Redis usado pela aplicação nunca é tocado.

Ponta a ponta com o processador

```
PROCESSOR_DATA_ROOT=/tmp/lume-e2e java -jar processor/target/processor.jar &
PROCESSOR_URL=http://localhost:8090 E2E_DATA_ROOT=/tmp/lume-e2e python -m pytest -m e2e
```

Sem o processador rodando, esses quatro testes são pulados.

Integração contínua

O workflow `.github/workflows/ci.yml` roda a cada push, em pull requests e uma vez por dia: compila o processador com Maven, sobe PostgreSQL e Redis como serviços, roda o Pytest completo (inclusive os testes de ponta a ponta com o jar recém-compilado) e constrói as imagens Docker.

Atividade diária

O workflow `.github/workflows/diario.yml` faz dois commits por dia, discretos, registrando uma linha no arquivo `.github/atividade.txt` (que guarda só as últimas 60 linhas). Commits de atividade não disparam o CI.

Turno	Horário (Brasília)	Reserva, se o primeiro falhar
1º turno	11:34	12:56
2º turno	21:12	22:43

1. No GitHub, abra **Settings → Secrets and variables → Actions**.
2. Crie os secrets `GIT_NAME` (seu nome no GitHub) e `GIT_EMAIL` (o e-mail *noreply* do GitHub, para não expor o pessoal).
3. Em **Actions → Atividade diária → Run workflow**, rode uma vez para testar.

Se o turno já foi registrado no dia, a execução reserva termina sem novo commit. O push tenta três vezes, com `git pull --rebase` entre elas.

CAPÍTULO 9

Solução de problemas

Sintoma	Causa provável	O que fazer
“port is already allocated” ao subir	Outra aplicação usa a porta 8080 ou 5432	Mude `APP_PORT` ou `DB_PORT` no .env e rode `docker compose up -d`
Página não abre por outro computador	Endereço fora de `APP_ALLOWED_HOSTS`	Acrescente o IP ou domínio e reinicie o web
Documento fica em “Falhou”	Arquivo recusado ou tentativas esgotadas	Veja o motivo no detalhe ou em Sistema; use “Ler o arquivo de novo”
“PDF protegido por senha”	PDF com senha de abertura	Remova a senha no leitor de PDF e envie de novo
“.doc antigos não são suportados”	Formato do Word 97–2003	Abra no Word e salve como .docx
Documentos parados em “Na fila”	Worker ou processador fora do ar	`docker compose ps`, depois `docker compose logs worker processor`; ao voltar, a fila anda sozinha
Sistema mostra “Processador fora do ar”	Contêiner parado ou sem memória	`docker compose restart processor`; documentos enormes pedem mais memória
IA “Falta configurar”	Provedor externo sem chave ou sem modelo	Preencha `AI_API_KEY` / `AI_MODEL` ou volte para `AI_PROVIDER=local`
Descrição em “Resumo automático” apesar da IA externa	A API falhou e o resumo local entrou no lugar	Veja o erro em Sistema → Últimas etapas e use “Gerar de novo”
Redis foi apagado ou reiniciado sem dados	Filas perdidas	Nada a fazer: os grupos são recriados e a varredura recoloca os documentos na fila
“Permission denied” em /data	Volume criado por outro usuário	As imagens usam UID/GID 10001; ajuste com `chown -R 10001:10001` no volume
“Reconectando…” no menu	WebSocket bloqueado pelo proxy	Libere o upgrade de conexão em `/ws/` no proxy reverso
Capa de um PDF ficou em branco	Primeiras páginas vazias ou só imagem transparente	O processador tenta as 3 primeiras; reenviar não muda — edite o PDF se quiser outra capa